

Implementation Report

Note: (A) $\rightarrow$ AdaIN; (G) $\rightarrow$ Gatys

1 Output Images

The model implementation, has not been vigorously fine tuned to present extremely accurate production quality results. Rather, they prove that style transfer is very well possible with the suggested methods (A) and (G).

The stylized images are presented in the attached Jupyter notebook files in the same directory. There is no clear winner among the two owing to the fact that they were not competitively optimized. But, the style transfer is clearly visible. In (G), it is observable that the first image produced is an abstract styling format and the content slowly creeps in and it is noteworthy to mention that the model was initially given the white noise image instead of the content image (which is reported to have easier convergence) just to visualize how the image develops. The output image of (A) also displays signs of style transfer (Color consistency, brush strokes, etc.) which can be drastically improved by changing the λ value (presently set to 10) and the α (blend factor - presently set to 0.2)

2 Content Preservation

Content preservation is best preserved in (A) from what I have observed. My explanation attempt:

1. (A) does not change/ transform the original content image directly from scratch.
2. It simply replaces the style statistics rather than completely relearning/optimizing it from scratch
3. On the contrary, (G) learns content by optimizing it with the loss function
4. In summary, major objects remain, edges stay similar, but small structures may slightly shift because pixels themselves being optimized in (G).
5. Whereas in (A), object locations stay almost identical, shapes are preserved well, only texture, colors and brush strokes change.

3 Stylization Strength

(A) controls Stylization strength on the basis of the blend coefficient. There is no dependence on the number of iterations and the stylization strength as during inference, we do not have from scratch optimization. (G) controls stylization with the help of the $\frac{\alpha}{\beta}$ ratio and is iterative, with more iterations yielding closer and finer style transfer outputs

4 Runtime and Implementation

1. (A) was relatively easy to understand but a bit difficult to implement owing to the elaborate and innumerable classes I had to define for preserving the modularity and reusability of the code. (G) was a completely different approach where it was not standard ML model, where we have a generic training loop, validation etc. It was directly optimizing the image, which was comparatively conceptually richer.
2. (G) had a greater runtime as it was optimizing during “inference”, while AdaIN was just inference of an already optimized model which took very less time in comparison